Tasarım Kavramları

Tasarım Her Yerdedir

Uluslararası Uzay İstasyonu

Haftaya planladığınız aile yemeği

Biz yazılım mimarisi ve tasarımına odaklansak da, genel tasarım kavramlarının çoğu yazılımda da rol oynar.

Bazı tanımlar

Tasarım: bilinçli, amaçlı planlama

Mühendislik: bilimsel ve matematiksel temelleri uygulayarak becerili veya sanatsal üretim

Zanaat: yetenekli meslek (yetenek deneyimden gelir)

Sanat: Beceri, tat ve hayal gücünü estetik nesnelerin üretiminde kullanma

Yazılım Tasarımı ve Programlama arasındaki farklar

Ölçek:

- Büyük bir çözüm gerektiren büyük bir problemle uğraşıyorsunuzdur.
- Programlarken, 100-1000-10000 satır kodla problemi çözersiniz.
- Uluslararası Uzay İstasyonunda 30 milyon satır kod var.

Fonksiyonel olmayan gereksinimlere önem vermek

- Programlarken, en fazla performansla ilgilenebilirsiniz.
- Tasarımda, işlevsel olmayan gereksinimler arası dengeyi sağlamalısınız.

Yazılım Tasarımı

Problemin işlevsel gereksinimlerini yerine getirirken işlevsel olmayan gereksinimleri de atlamadan yazılım üretimi

Birşeylerden ödün vermek gerekecek ama hangisinden ve nasıl?
- Performans ve Kaynak Kullanımı?

Performans için tasarım yapsaydınız, dizi mi kullaırsınız nesne koleksiyonu mu?
- Hava tahmininde komşuluk gridi değişirse?

Mimari Tasarım:
- Bir yazılımın modül veya bileşenlerine davranışsal yönleriyle belirlenmiş sorumlulukları atama işlemi
- Program parçaları bileşenler halinde birleştirilir, her bileşene davranış şeklinde sorumlulukları yüklenir, bileşenlerin nasıl etkileşime geçeceği belirlenir.

Detaylı Tasarım:
- Mimari tasarımda belirlenen davranışların teker teker bileşenlere veri yapıları ve algoritmaları da düşünülerek tanımlanması

Detaylı Tasarım Dili

Pseudo Code (sözde kod) Program Tasarım Dili (PDL)
- Anahtar kelimeler, doğal dilde serbest ifadeler, veri tanımları, alt program tanımı ve çağrıları

Yapısal Programlama
- Sıralı ifadeler, koşullar, döngüler, bölütleme (alt program)

Akış Diyagramı
- Yönlü graflar, düğümler işlem birimleri, bağlantılar veri akışları

Karar Tabloları
- Kurallar, koşullar, aksiyonlar

Yazılım Tasarımı Yaklaşımları

Bir sistemi en iyi nasıl yapılandırırız sorusunu cevaplayanlar
- Nesneye yönelik tasarım
- Yapısal tasarım
- Rol tabanlı tasarım

Belirli bir sınıfa ait yazılımlara odaklananlar
- Gerçek zamanlı sistemler

Uygulamanın sadece belirli bir kısmına odaklananlar
- UI/UX tasarımı

Yazılım Tasarımı Yaklaşımları

Tüm yaklaşımlarda ortak olan 3 yön:
- Tasarım yöntemi:
  - Tasarımcıların bir problemi çözmek için uyguladıkları sistematik ve sıralı işlem adımları.
  - Genellikle problemi göstermek için belirli bir yol önerir.
  - Örneğin Nesneye yönelik tasarımda, problem işbirliği yapan nesneler şeklinde tanımlanır.
  - Yöntem genel olarak tüm takıma düşünce ve aktivitelerini belirli şekilde yapmaları için disiplin sağlar
- Tasarım gösterimi
  - Görsel veya yazılı ifadelerle tasarımı ifade etme
- Tasarım geçerleme

Yazılım Tasarım İkilemleri

Mimar/tasarımcı olarak kararlar vermeniz gerekir.
- İşleri yukarıdan aşağıya doğru mu, aşağıdan yukarı doğru mu, içten dışa doğru mu yapacaksınız?
- Yordam ve fonksiyonlar mı düşüneceksiniz, kavramlar ve davranışlar mı?

Yazılım üretiminde kaçınılmaz sorun: Tasarım zaman alır.
- Uzun dönem yönetilebilir yapı mı, kısa süreli takvimli işi yetiştirmek mi?

Hangi araçlar kullanılacak?

Tasarım gözden geçirme (geçerleme)

Tasarım yönteminiz var.

Tasarım gösteriminizi de seçtiniz.

Tasarımınızı yaptınız.

Bir takım, gereksinimlerle uyumlu olup olmadığını gözden geçirir.

Bu aşamada gözden geçirmeye gerek var mı?
- Zaten programı gerçekleyeceğiz, (pek çoğu otomatikleşmiş) testler yapacağız?
- En sonda gözden geçirsek?

Problemi ne kadar erken tespit ederseniz, o kadar ucuz ve hızlı çözersiniz.

Tasarım Geçerleme

Bir takım tarafından tasarım gözden geçirilir. Kullandığınız araçlar da bir takım kontrolleri sizin için yapabilir.

Önemli hususlar:

Geçerleyenlerin bağımsızlığı
- Tasarlayan ve geçerleyen aynı kişiler olursa, belirli şeyleri gözden kaçırabilirler
- Tasarlarken düşünmedikleri detayları, geçerlemede de düşünemezler.

Tasarım yöntemine bağımlılık
- Yapısal tasarımda tasarım sonuçlarıyla metrikleri eşleştiren kurallar kümesi mevcuttur

Tasarlarken geçerleme / Bitirince geçerleme
- Günlük veya haftalık olarak yaptıklarınızı gözden geçirip düzeltmek
- Kilometre taşına kadar üretip, geçerleme ve düzeltmeleri yapmak

Diğer tasarım unsurları

Mimari ve detaylı tasarımın sınırları nerededir?

İşlevsel ve işlevsel olmayan gereksinimlere ne kadar ağırlık verilmelidir?

"Ne" ve "nasıl" dengesi nasıl kurulacak?
- Tanımlama (ne)
- Tasarlama (nasıl)

Uygulama özelinde tasarım işlemlerini etkileyen faktörler nelerdir?
- Belirli kaynak ve önceden hazırladığımız çözümleri kullanmak isteyebiliriz

Tasarım belgelendirme

Büyük sistemler karmaşıktır.

Ölçek ve karmaşıklık, iyi belgelendirme ihtiyacını arttırır.

Tüm gücümüzü geliştirme için kullanmış olalım.
- Sistem bir süre iyi çalışacaktır.
- Sonrasında bakıma girecektir.
- Tasarım belgelendirme, taraflar arası iletişimi kolaylaştırır

Formal belgeler ya da basit notlar, hatta sunumlar bile olabilir.

Geleneksel Tasarım Belgeleri

Sistemi bir araya getiren bileşenler
- Sorumlulukları
- Veri akışları

Kontrol akışı

Performans kriterleri

Kaynak kullanımı (hafıza, harici birimler, bant genişliği, …)

Bunlar yetmiyorsa, IEEE 1016 standardı
- Tasarımcı kim, parçalar arası bağımlılıklar neler, gizli varsayımlar var mı, işlevsel olmayan gereksinimler arası ödünleşmelere nasıl karar verdiniz, kullanıcı/teknoloji/donanım varsayımlarınız neler, …

Tasarım gerekçelendirme

Bu kadar detaylı bilgileri bir araya getirerek tasarım gerekçeleri oluşturulur

Tasarım çözümümüzde neler yaptık, neden yaptık

- Ör: Hız ve boyut arasında nasıl bir seçim yaptık, neye göre yaptık

Ne kadar açık sebeplere dayalı seçimler yaparsak, ileride sistemin bakımını yapacak taraflara o kadar kolaylık sağlarız.

Tasarımda Anahtar Kavramlar

Kavramsal Bütünlük

- Descartes: Farklı ellerden çıkan çok parçalı işlerde mükemmellik, tek bir ustanın elinden çıkana oranla çok az görünür.
- Fred Brooks (The Mythical Man-Month): Bağımsız ve koordine olmamış kavramlar yerine bir tasarım fikir kümesinden çıkmak kaydıyla, alışılmadık özellikler ve gelişmeler sağlayan sistemler iyidir

Az Bağımlılık

- Sisteminizin bileşenleri birbirine bağımlıdır. Başarılı çalışma için iki bileşenin mümkün olduğunca az bağımlı olmaları istenir.
- Çok bağımlı olurlarsa, birindeki değişiklik diğerini de değiştirmenizi gerektirir.

İyi Uyum

- Bir bileşenin belirli bir amacı ya da hizmeti olması iyidir.
- İyi uyuma sahip birimleri tekrar kullanma olasılığı daha yüksektir.

Java paketleri az bağımlılığı mı iyi uyumu mu destekler?

Java sınıf kalıtımı bağımlılığı arttırır mı azaltır mı?

Tasarımda Anahtar Kavramlar

Bilgi Gizleme (David Parnas)

- Belirli bir modülün yapabildiklerini, soyut bir arayüz arkasında saklamak. Kullananlar sadece görünen arayüzü bilebilir.
- Gerçekleştirme detaylarımızı değiştirebilmemizi sağlar.
- Erişim gizliliği (donanıma, veritabanına ,sunucuya, …) sağlar

Soyutlama (abstraction)

- Karmaşık sistemdeki problemleri ayrıştırma, belirli noktalara odaklanma
- Analizi tanımlarken tasarım detaylarından soyutlanmak
- Programlama dillerinde dizi, struct, nesne gibi yapıların içeriği soyutlaması
- Nesneye yönelik genelleştirme: özel durumlar yerine sınıf davranışı
- Parametreler: Farklı tipler ve detaylar yerine sadece isimlerle kullanım

Tasarımda Anahtar Kavramlar

Estetik

- Aquinas (James Joyce çevirisi): Güzellik için üç şey gerekir: Bütünlük, ahenk, saflık
- Yazılım karşılıkları: Bütünlük (completeness), tutarlılık (consistency), kavramsal bütünlük (conceptual integrity)
- Pascal: Bu mektubu çok uzun yazdım çünkü daha kısa yapabilecek kadar vaktim yoktu.
- Karmaşık gereksinimleri yerine getirecek basit ve etkin bir çözüm geliştirmek için zaman ve enerji harcamanız gerekir.

Tasarım Felsefesi (Piete En)

- Yazılım tasarımı dört filozof ile ilişkilendirilir.
- Descartes: Analitik geometri. Analiz, model kullanımı
- Marx: Sosyal etki. Kullanıcı merkezli tasarım, tasarımın sosyal etkileri (hatta kullanıcıları tasarıma dahil etmek)
- Martin Heidegger: Hedeflere ulaşmak için araç kullanımı. CASE, IDE
- Wittgenstein: Dil oyunları. Bir kelimenin keşfi, bir kavramla ilgili düşünmenizi sağlar. Desktop, folder, trash, client/server, icon, firewall…

Tasarım

Yazılım geliştirmenin en yaratıcı kısmıdır.

Sistemin genel kalitesi, üretilen tasarımlara direk bağımlıdır.

Tasarım kalitesini yansıtan önemli bir gösterge, tasarım ekibinin benzer projelerdeki deneyimidir.

UML Diyagramları